

Code:

```
class FixedSizeQueue:
```

```
    def __init__(self, capacity=4):
```

```
        self.queue = [None] * capacity
```

```
        self.capacity = capacity
```

```
        self.front = 0
```

```
        self.rear = 0
```

```
        self.size = 0
```

```
    def enqueue(self, value):
```

```
        if self.isFull():
```

```
            print("Queue is full. Cannot enqueue:", value)
```

```
        else:
```

```
            self.queue[self.rear] = value
```

```
            self.rear = (self.rear + 1) % self.capacity
```

```
            self.size += 1
```

```
            print(f"Enqueued: {value}")
```

```
    def dequeue(self):
```

```
        if self.isEmpty():
```

```
            print("Queue is empty. Cannot dequeue.")
```

```
        else:
```

```
            value = self.queue[self.front]
```

```
            self.queue[self.front] = None
```

```
            self.front = (self.front + 1) % self.capacity
```

```
            self.size -= 1
```

```
print(f"Dequeued: {value}")
```

```
def peek(self):
```

```
    if self.isEmpty():
```

```
        print("Queue is empty. Nothing to peek.")
```

```
    else:
```

```
        print(f"Front element: {self.queue[self.front]}")
```

```
def isEmpty(self):
```

```
    return self.size == 0
```

```
def isFull(self):
```

```
    return self.size == self.capacity
```

```
def print_queue(self, label="queue (array)":
```

```
    if self.isEmpty():
```

```
        print(f"{label}: (empty)")
```

```
    else:
```

```
        index = self.front
```

```
        items = []
```

```
        for _ in range(self.size):
```

```
            items.append(str(self.queue[index]))
```

```
            index = (index + 1) % self.capacity
```

```
        print(f"{label}: {' '.join(items)}")
```

```
queue = FixedSizeQueue()
```

```
queue.enqueue(1)
```

```
queue.enqueue(2)
```

```
queue.enqueue(3)
```

```
queue.print_queue("Queue")
```

```
queue.dequeue()
```

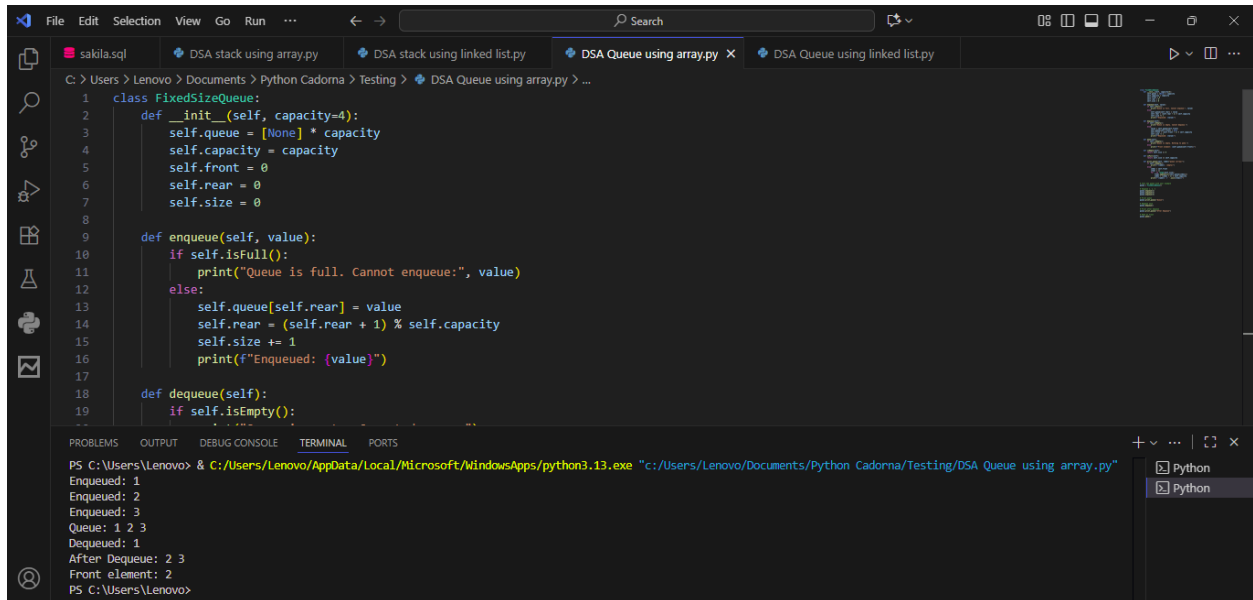
```
queue.print_queue("After Dequeue")
```

```
queue.peek()
```

Programiz Link:

<https://www.programiz.com/online-compiler/3hbPl15ouqZg2>

Sample Output:



The screenshot displays a Visual Studio Code editor window with a Python script for a `FixedSizeQueue` class. The script is located at `C:\Users\Lenovo\Documents\Python Cadorna\Testing\DSA Queue using array.py`. The code defines a class with methods for initialization, enqueue, and dequeue. The terminal output shows the execution of the script, which enqueues the values 1, 2, and 3, then dequeues the first element (1), resulting in a queue containing [2, 3] and a front element of 2.

```
C:\Users\Lenovo\Documents\Python Cadorna\Testing\DSA Queue using array.py > ...
1 class FixedSizeQueue:
2     def __init__(self, capacity=4):
3         self.queue = [None] * capacity
4         self.capacity = capacity
5         self.front = 0
6         self.rear = 0
7         self.size = 0
8
9     def enqueue(self, value):
10        if self.isFull():
11            print("Queue is full. Cannot enqueue:", value)
12        else:
13            self.queue[self.rear] = value
14            self.rear = (self.rear + 1) % self.capacity
15            self.size += 1
16            print(f"Enqueued: {value}")
17
18    def dequeue(self):
19        if self.isEmpty():
20            print("Queue is empty. Cannot dequeue.")
21        else:
22            self.front = (self.front + 1) % self.capacity
23            self.size -= 1
24            print(f"Dequeued: {self.queue[self.front-1]}")
25
26    def isFull(self):
27        return self.size == self.capacity
28
29    def isEmpty(self):
30        return self.size == 0
31
32    def display(self):
33        print("Queue: ", end="")
34        for i in range(self.front, self.rear+1):
35            print(self.queue[i], " ", end="")
36        print()
37
38    def __str__(self):
39        return f"Queue: {self.queue[self.front:self.rear+1]}"
40
41 if __name__ == '__main__':
42     q = FixedSizeQueue(4)
43     q.enqueue(1)
44     q.enqueue(2)
45     q.enqueue(3)
46     q.dequeue()
47     q.display()
48     print(f"Front element: {q.queue[q.front]}")
49     print(f"Rear element: {q.queue[q.rear]}")
50     print(f"Size: {q.size}")
51     print(f"Capacity: {q.capacity}")
52     print(f"Front: {q.front}")
53     print(f"Rear: {q.rear}")
54     print(f"Is Full: {q.isFull()}")
55     print(f"Is Empty: {q.isEmpty()}")
56     print(f"Queue: {q}")
57     print(f"Queue: {str(q)}")
58     print(f"Queue: {list(q.queue)}")
59     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
60     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
61     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
62     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
63     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
64     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
65     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
66     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
67     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
68     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
69     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
70     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
71     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
72     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
73     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
74     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
75     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
76     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
77     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
78     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
79     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
80     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
81     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
82     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
83     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
84     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
85     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
86     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
87     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
88     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
89     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
90     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
91     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
92     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
93     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
94     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
95     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
96     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
97     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
98     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
99     print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
100    print(f"Queue: {list(q.queue[q.front:q.rear+1])}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Lenovo/Documents/Python Cadorna/Testing/DSA Queue using array.py"
Enqueued: 1
Enqueued: 2
Enqueued: 3
Queue: 1 2 3
Dequeued: 1
After Dequeue: 2 3
Front element: 2
PS C:\Users\Lenovo>
```